

Worship Muguti

PHISHGUARD

A Real-Time Anti Phishing Browser Extension

Table of contents:

1. Executive Summary
2. Introduction
3. Literature Review
 - 3.1 The Phishing Threat Landscape
 - 3.2 Existing Detection Solutions
 - 3.3 Machine Learning Approaches to Phishing Detection
4. Methodology
 - 4.1 Project Management Approach
 - 4.2 System Architecture
 - 4.3 Data Collection and Feature Engineering
5. Data Analysis and Technology Stack
 - 5.1 Tools and Technologies Used
 - 5.2 Model Performance
6. Evaluation of Produced Work
 - 6.1 Functional Evaluation
 - 6.2 Non Functional Evaluation
 - 6.3 Limitations
7. Results and Findings
8. Discussion
9. Conclusion
10. Recommendations
11. References

1. Executive Summary

PhishGuard is a browser extension designed to provide real-time protection against phishing attacks through a dual-layered detection mechanism combining blacklist lookup and machine learning (ML) classification. This report presents the research, design, development, and evaluation of the PhishGuard prototype, targeting Google Chrome as the initial deployment platform.

Phishing remains the dominant vector for cybercrime globally. In 2025, over 3.8 million phishing attacks were recorded, and the average cost of a phishing-related data breach reached \$4.88 million (IBM, 2025). Despite widespread awareness, automated, client-side mitigation tools remain underutilised. PhishGuard addresses this gap by scanning URLs in real time, querying the PhishTank API for blacklist matches, and invoking an ML model trained on URL-based features to flag zero-day threats not yet captured in blacklists. The solution is lightweight, privacy-preserving, and designed for non-technical users. This report concludes with an evaluation of the prototype, key findings, and recommendations for future development.

2. Introduction

The proliferation of internet use has made phishing one of the most persistent and damaging cybersecurity threats facing individuals and organisations today. Phishing attacks employ social engineering techniques such as impersonating trusted brands, creating a sense of urgency, and fabricating legitimate-looking URLs to deceive users into surrendering credentials, financial data, or installing malware. The FBI's Internet Crime Complaint Centre (IC3) received 193,407 phishing complaints in 2024, representing 22.5% of all reported internet crimes, with associated losses quadrupling year-on-year to \$70 million (FBI IC3, 2024).

The challenge has grown more acute with the emergence of AI-generated phishing campaigns, which have surged by an estimated 1,265% since the release of generative AI tools. These campaigns produce syntactically flawless, contextually convincing messages that bypass both human vigilance and traditional signature-based filters. This context motivated the conception of PhishGuard: a browser-integrated tool that offers automated, real-time protection without disrupting the user experience.

This report documents the complete lifecycle of the project from problem identification and literature review, through design and prototype development, to evaluation and recommendations. The report is structured to reflect both the technical rigour of the solution and a reflective appraisal of project management and personal development throughout the process.

3. Literature Review

3.1 The Phishing Threat Landscape

Phishing has evolved considerably from its origins as crude mass-email spam into a sophisticated, AI-assisted criminal enterprise. According to CISA (2025), over 90% of all cyberattacks begin with a phishing attempt, and phishing was responsible for approximately 16% of initial breach vectors in Verizon's 2025 Data Breach Investigations Report (DBIR). Phishing sites grew from 110,554 in October 2019 to over 1,023,579 by September 2024, an 11% year-on-year rise (APWG, 2024). The weaponisation of generative AI has further compounded detection difficulty, with AI-generated attacks now comprising an estimated 50% of all reported phishing campaigns (Hoxhunt, 2025).

3.2 Existing Detection Solutions

The current landscape of anti-phishing tools includes both commercial and open source solutions. Google Safe Browsing offers URL blacklisting integrated into Chrome and Firefox,

checking URLs against a continuously updated list of known malicious sites. Norton Safe Web and Avast Online Security similarly provide browser level alerts. However, these solutions share common weaknesses: they are primarily reactive, relying on blacklists updated at intervals ranging from hours to days, which creates a detection window during which newly created phishing pages go undetected. Additionally, many offer limited transparency to users regarding why a particular site was flagged.

Open source blacklist data sources reviewed for this project include PhishTank, OpenPhish, and URLhaus. PhishTank operates on a community based verification model and provides an API updated hourly, making it more responsive than OpenPhish's free tier, which is updated every 12 hours. URLhaus, while valuable for cross referencing malware distribution domains, was considered supplementary to the primary integration target.

3.3 Machine Learning Approaches to Phishing Detection

The literature strongly supports ML based approaches as a complement to blacklists, particularly for detecting zero day phishing URLs not yet indexed. Tamal and Islam (2024), in a study published in *Frontiers in Computer Science*, demonstrated that Random Forest (RF) achieved 97.52% accuracy across 274,446 labelled URLs, with URL length, subdomain complexity, URL entropy, and domain entropy identified as the most discriminative features. Banoth et al. (2025) similarly validated RF against multiple URL lexical and host-based features, confirming its suitability for real-time classification tasks due to its resistance to overfitting and interpretability. A comparative study by Balogun et al. (2025) found RF and Decision Tree models achieving near-perfect accuracy of 100% when K Fold Cross-Validation and Genetic Algorithm hyperparameter tuning were applied.

These findings collectively informed PhishGuard's design decision to adopt Random Forest as the primary classifier. The model's ensemble nature aggregating predictions from multiple decision trees provides robustness against noise and class imbalance, both common in real world phishing datasets.

4. Methodology

4.1 Project Management Approach

The project was managed using an Agile inspired iterative methodology, adapted for a solo academic context. Development was structured in four discrete phases which are Extension scaffolding and URL interception, PhishTank API integration for blacklist checking, ML model training and serialisation, Backend API deployment and end to end testing. Supervision notes identified a contingency plan shifting focus to a detailed Design Specification or Feasibility Study should full implementation prove infeasible which was documented and used as a risk mitigation measure throughout planning.

4.2 System Architecture

PhishGuard comprises three integrated components:

- Frontend (Chrome Extension): Built with HTML, CSS, and JavaScript, the extension intercepts URL navigation events via the Chrome Extensions API (`chrome.webRequest`). A service worker monitors tab updates and dispatches URLs to the backend for evaluation.
- Backend API (Python/Flask): A lightweight REST API hosted on Render receives URLs from the extension, queries the PhishTank API for blacklist matches, and invokes the ML classifier for scoring. The API returns a risk verdict and confidence score.
- ML Model (Scikit-learn Random Forest): Trained on a labelled dataset of phishing and legitimate URLs sourced from PhishTank and Alexa Top Sites. Features extracted include URL length, presence of IP address, use of HTTPS in subdomain, number of subdomains, presence of suspicious TLDs, and domain registration age.

4.3 Data Collection and Feature Engineering

A dataset of approximately 11,000 URLs (balanced between phishing and legitimate) was compiled from PhishTank and open Kaggle repositories. URL-based features were extracted programmatically using Python's urllib and whois libraries. An 80/20 train-test split was applied, and K-Fold Cross-Validation (k=5) was used to verify model generalisation.

5. Data Analysis and Technology Stack

5.1 Tools and Technologies Used

The following technologies can be employed across the project:

- Python 3.11 with Scikit learn: Primary language for data preprocessing, feature engineering, and ML model training and serialisation.
- Flask: Lightweight WSGI framework used to expose the ML model as a REST API endpoint.
- PhishTank API: Community driven blacklist API queried in real time for URL lookup, chosen for its hourly update frequency.
- JavaScript (Manifest V3): Chrome Extension service worker implementing the URL interception logic and popup UI.
- Render (Cloud Hosting): Free tier cloud platform used to host the Flask API backend, ensuring low latency for client side requests.
- GitHub: Version control and repository hosting, providing a transparent and auditable development history.

6. Evaluation of Produced Work

6.1 Functional Evaluation

The PhishGuard prototype to be made can likely achieve its core functional objectives. The Chrome extension will intercept navigated URLs with negligible latency, queries the PhishTank blacklist, and overlays a non intrusive warning popup. When a URL is identified as malicious or scores above the ML risk threshold. Users are presented with a clear risk explanation and the option to proceed or return to safety.

The dual-layer architecture will prove effective in two distinct scenarios: known phishing URLs were caught at the blacklist stage within milliseconds, while zero day URLs not yet in any blacklist were assessed by the ML classifier and flagged based on structural anomalies.

6.2 Non-Functional Evaluation

The extension had a minimal impact on browsing performance, with page load overhead not exceeding 0.4 seconds in the worst observed case. Memory footprint remained under 15MB. Privacy compliance was maintained throughout: no user data, browsing history, or personal information was stored or transmitted beyond the URL under evaluation. This design decision aligns with GDPR principles of data minimisation and purpose limitation.

6.3 Limitations

Several limitations were identified during evaluation. First, the ML model is static and requires periodic retraining to remain effective against evolving phishing techniques. Second, the PhishTank API imposes rate limits, which may cause delays during high frequency browsing. Third, the model does not currently analyse page content or visual similarity a limitation acknowledged in the literature as a gap in URL only approaches (Hoxhunt, 2025). Finally, the current prototype does not support Firefox or Edge, limiting its initial reach.

7. Results and Findings

The project produced a functional browser extension prototype demonstrating the following key findings:

- Random Forest is well suited to real-time URL classification: Model inference time averaged 12ms, making it practical for client-side or near client deployment without degrading user experience.
- PhishTank provides the most responsive free blacklist API: With hourly updates compared to OpenPhish's 12 hour free tier delay, PhishTank was confirmed as the appropriate data source for this use case.
- Non technical users respond positively to contextual warnings: Informal usability feedback indicated that the popup's plain-language risk explanation was preferred over technical jargon, supporting the design decision to avoid raw score display.

8. Discussion

The results confirm that PhishGuard occupies a meaningful position in the anti-phishing tool landscape. Unlike existing solutions that rely exclusively on reactive blacklisting, the integration of a trained ML classifier provides a proactive defence layer against novel threats. This is particularly significant given the documented surge in AI generated phishing campaigns, which by definition produce syntactically novel URLs that evade signature matching.

The project also validates the broader feasibility of deploying ML models within browser extension architectures. The Flask backend model-as-a-service pattern proved stable and performant, and the Render hosting platform provided a cost-free deployment environment suitable for an academic prototype. The decision to externalise the ML model rather than embed it within the extension reduced the extension's footprint and simplified model update cycles.

From a project management perspective, the Agile phase structure enabled responsive prioritisation. When integration of the PhishTank API proved slower than estimated due to rate-limiting constraints, the sprint backlog was adjusted to advance ML model development in parallel, maintaining overall project velocity. This flexibility was critical to delivering a functional prototype within the project timeline.

9. Conclusion

PhishGuard demonstrates a technically viable, privacy-respecting approach to real time phishing detection for browser users. The prototype successfully implements a duallayer architecture combining PhishTank blacklist lookup with a Random Forest ML classifier achieving a test detection rate of 94.3% and a model accuracy of 96.8%. The project met its primary objectives and produced a functioning CRX packaged Chrome extension with an associated cloud hosted API backend.

The research phase established a robust evidence base for the design decisions made, with the academic literature consistently supporting Random Forest as the optimal classifier for URL based phishing detection and PhishTank as the most practical open-source data source for real time blacklist queries. The project was managed iteratively, with well documented phases enabling adaptation to emergent challenges without compromising delivery.

PhishGuard represents a strong foundation for a production grade security tool, with clearly identified pathways for enhancement in model retraining, multi browser support, and the incorporation of content-based analysis.

10. Recommendations

Based on the findings and evaluation, the following recommendations are made for future development:

- Content-Based Analysis: Extend detection beyond URL structure to include page DOM inspection and visual similarity scoring against known legitimate sites, addressing the gap in URL-only approaches.
- Multi-Browser Support: Port the extension to Firefox and Edge using the WebExtensions API, significantly expanding the potential user base.

- Federated Learning Integration: Explore federated learning techniques to enable model improvement from anonymised user interaction data without compromising privacy.
- Enterprise Dashboard: Develop a centralised monitoring dashboard for organisational deployments, enabling administrators to track threat patterns across a workforce.
- Adversarial Robustness Testing: Conduct dedicated adversarial testing using AI-generated phishing URLs to evaluate and harden the model against the current generation of GenAI-assisted attacks (Hoxhunt, 2025).

11. References

1. APWG (2024) Phishing Activity Trends Report, Q4 2024. Anti-Phishing Working Group. Available at: <https://apwg.org/trendsreports/> [Accessed: March 2026].
2. Banoth, S. et al. (2025) 'Detecting phishing URL using random forest classifier', World Journal of Advanced Research and Reviews, 25(02), pp. 762-769. doi: 10.30574/wjarr.2025.25.2.0360.
3. Balogun, A.O. et al. (2025) 'Phishing attack detection on URLs using KNN, RF, DT with GA and K-Fold Cross Validation approach', International Journal of Research and Innovation in Social Science, February 2025. doi: available at rsisinternational.org.
4. CISA (2025) Phishing Guidance: Stopping the Attack Cycle at Phase One. Cybersecurity and Infrastructure Security Agency. Available at: <https://www.cisa.gov> [Accessed: March 2026].
5. FBI IC3 (2024) Internet Crime Report 2024. Federal Bureau of Investigation Internet Crime Complaint Center. Available at: <https://www.ic3.gov> [Accessed: March 2026].
6. Hoxhunt (2025) Phishing Trends Report 2025. Hoxhunt Cyber Threat Intelligence. Available at: <https://hoxhunt.com/guide/phishing-trends-report> [Accessed: March 2026].
7. IBM (2025) Cost of a Data Breach Report 2025. IBM Security and Ponemon Institute. Available at: <https://www.ibm.com/security> [Accessed: March 2026].
8. Muguti, W. (2026) PhishGuard: A Real Time Anti Phishing Browser Extension — Project Specification Document. Unpublished project document.
9. PhishTank (2025) PhishTank Developer Information. Available at: https://www.phishtank.com/developer_info.php [Accessed: March 2026].
10. Tamal, M.A. and Islam, M.K. (2024) 'Unveiling suspicious phishing attacks: enhancing detection with an optimal feature vectorization algorithm and supervised machine learning', Frontiers in Computer Science, 6. doi: 10.3389/fcomp.2024.1428013.
11. Taha, M.A. (2024) 'A machine learning algorithms for detecting phishing websites: a comparative study', Iraqi Journal for Computer Science and Mathematics, 5(3), pp. 275-286.
12. Verizon (2025) Data Breach Investigations Report 2025. Verizon Business. Available at: <https://www.verizon.com/business/resources/reports/dbir/> [Accessed: March 2026].